Detecting Prediction Collapse Caused by Pooled Feature Normalization in Space-Trajectory Screening

Harsha Vardan M Sakamuri[1], Rohit Michael Raj[1], Valarmathi P.[1*]

[1]*School of Computer Science and Engineering (SCOPE), Vellore Institute of Technology, Chennai 600127, India*

*Corresponding author: valarmathi.sudhakar@vit.ac.in

Abstract

Monte Carlo trajectory campaigns spend much of their computing time on runs that are already sure to fail, and machine learning screens that stop such runs early are an attractive way to save that time. These screens depend on routine preprocessing choices. We show that one common choice, fitting a single feature scaler to data pooled across target groups with very different scales, can make a Transformer screen give almost the same prediction for every mission of a target. On Venus the spread of the predictions falls to 3.11e-05 and the AUC falls from 0.9999 to 0.6037, while an XGBoost baseline on the same arrays stays near perfect and a pooled score shows the damage only in part. Across five seeds the collapse occurs on four. We propose a simple check, the spread of the predictions within each group, which needs no labels at run time. With a threshold set on validation runs it flags the four collapsed runs out of 60 and no working run, and it raises no alarm on a second dataset of turbofan engines. The work is relevant to any learned screen trained on grouped data.

Keywords: feature normalization, prediction collapse, model diagnostics, deep learning, Monte Carlo simulation, trajectory screening

1 Introduction

Designing a space mission is not done with a single trajectory. Engineers run large Monte Carlo campaigns, in which thousands of trajectories are simulated with small random changes to the launch and the engine burns, to see how often the mission still reaches its target. NASA's TESS mission, for example, used a large dispersion campaign of this kind to set its trajectory design[1]. Every one of these trajectories has to be propagated, which means that a numerical integrator follows the spacecraft forward in time under the gravity of the Sun and the planets. For transfers to the outer planets one run covers years of simulated flight, so a campaign is expensive.

A large share of that cost is wasted. In our campaigns most of the compute is used on runs that were already sure to fail when they left the parking orbit. One line of work makes each propagation cheaper[2,3]. Another replaces part of an expensive simulation with a learned surrogate[4]. A third, simpler idea is to watch the first part of each trajectory and stop the runs that are going to fail. We call such a model a screen. Machine learning is a natural fit for it, and learned models are already used widely in astrodynamics[5,6].

A learned screen is more than a network. It is a pipeline, and one of its quiet steps is feature scaling. Before the data reach the network, every feature is shifted and divided by a typical size, so that features measured in very different units become comparable. This step is old and well understood[7,8], and it is usually done with one line of code. In our data, however, the features come from seven different target planets, and their ranges differ by orders of magnitude. If one scaler is fitted to all targets of a group together, the large differences between targets set the scale, and the small differences between missions of the same target, which are the ones that decide success or failure, are squeezed close to zero.

This is what happened in our screen. The model ended up giving a single, almost constant probability of failure to every mission of a target. That a pooled scaler can do this is not surprising. The difficulty is that it is easy to miss. A pooled score shows the damage only in part: in the controlled ablation of Section 4, the AUC over the three targets of a group falls from 0.9995 to 0.875, while the AUC of the failed target alone is 0.687. A gradient boosted tree trained on the same scaled arrays, which is the usual sanity check, scores almost perfectly, because trees split on the order of the values and the rescaling does not change that order. Both checks can leave the engineer with the impression that the data are fine and that only the network needs tuning.

The failure can be described as a preprocessing error, and its remedy, scaling each group separately, is simple. The contribution of this paper is not the remedy. It is a careful description of a failure that a pooled score shows only in part and that the tested scale-invariant baseline does not show, together with a check on the spread of the predictions that does show it. The problem is not specific to astrodynamics. Any learned model trained on grouped data, such as engines running under different operating conditions or sensors at different sites, can meet the same failure.

The main contributions of this work are as follows.

- A controlled decomposition of the collapse. Keeping the model, the data split and the training fixed, we change only how the scaling statistics are computed, and show that pooling over time within one target does little harm, while pooling across targets can make the model output a constant.
- An analysis of why the usual checks miss it. A tree baseline on the same input is not affected by this kind of rescaling, and a pooled score averages the failed target with the healthy ones.
- A simple diagnostic, the spread of the predictions within each group, with a fully stated evaluation protocol, a sensitivity analysis, a comparison with other summaries of the predictions, statistical tests, and checks on batch size and calibration.
- A second, public dataset (NASA turbofan engines) on which the same pooling shrinks the signal without breaking the model, and an economic analysis that places the learned screen in its operational context.

The rest of the paper is organized as follows. Section 2 reviews related work on machine learning in astrodynamics, on feature normalization, and on checks for degenerate models. Section 3 describes the simulated data, the scaling settings, the models, the diagnostic and the statistical methods. Section 4 reports the experiments. Section 5 discusses what the results mean for mission assurance and what they do not show. Section 6 concludes the paper.

2 Literature review

2.1 Machine learning in astrodynamics and simulation screening

Neural networks have been applied successfully in astrodynamics for trajectory optimization and for approximating optimal control[5,6,9], for estimating the cost of a transfer without a complete propagation[10,11], and for learning robust low-thrust guidance policies that are then certified through Monte Carlo campaigns[12]. Learned models are also used to improve orbit predictions from publicly available tracking data[13], and reduced models make the propagation of orbit uncertainty cheaper than plain sampling[14].

Spacecraft telemetry is another large area. Learned models detect anomalies and faults in telemetry streams[15], including attention-based models of the same family as ours[16], and careful pipelines based on recurrent networks perform well when every channel is processed correctly[17]. Recent work also shows how easily the reported performance of such detectors is overestimated, and that simpler methods can match complex ones once the evaluation is done

carefully[18]. A public benchmark for anomaly detection in satellite telemetry is now available[19].

Screening itself, where a learned model decides which of a very large number of candidate cases get a full analysis, is already established. An ESA competition used real conjunction data to rank collision risks[20,21], and a benchmark compared deep approaches with the classical filters they would replace in conjunction screening[22]. The last one is the closest setting to ours: a screening task on a deterministic astrodynamics simulator, measured against non-deep baselines. We could not find an article that considers early termination of individual Monte Carlo trajectory runs, as we do in Section 4.7. The closest work makes each propagation cheaper rather than deciding which propagations to start[2,3], or uses learned surrogates in place of parts of a simulation[4].

2.2 Feature normalization in deep learning and time series

Normalization is central to training deep networks, and there are standard layers for it (batch, layer, group and instance normalization)[8,23–26]. It has long been known that the ability of a network to train, and not only the speed at which it trains, depends on how the inputs are scaled[7]. These layers also make assumptions about their input. They assume that the feature statistics are stationary, or that they are comparable within each pooling axis. When data from several sources with very different ranges are measured with the same scaler, that assumption is broken before the first layer runs.

Normalization has also been studied for time series on its own. ST-Norm splits it into a temporal part and a spatial part for forecasting models[27], and Deep Adaptive Input Normalization learns the scaling from the data[28]. Those papers aim at a more accurate model. We ask a different question: how can a poor scaling choice break a model without anyone noticing? What we add is a controlled decomposition showing that pooling in general does not do the damage, only pooling across groups, and the observation that the resulting model is not a weak model but a constant one.

2.3 Shortcut learning, degenerate models and checks on machine learning pipelines

When a shortcut is available, a network easily learns the easy signal and takes the shortcut instead of the intended structure[29,30]. Work on Clever Hans behavior shows that models can score well for reasons that have nothing to do with the task[31]. When groups have very different scales, there is a particularly clean shortcut: the distance between the group centers. A model of this type can learn the difference between groups, and can reach a high AUC on a

mixed validation set while performing no better inside the individual groups. The failure is also not a modeling problem but a pipeline problem, of the kind that lives in shared preprocessing code that no single model owns[32].

Tree ensembles perform well on tabular and physical data, often matching or beating deep models[33–37], and many practitioners use a tree as a baseline for that reason. Their splits are axis aligned and based on thresholds, so a monotone rescaling of a feature keeps the available ordering. That useful invariance is exactly why the tested XGBoost baseline misses this particular failure when it receives the same transformed arrays. It does not mean that tree baselines are unable to find preprocessing errors in general.

The word collapse also needs care. Neural collapse describes the geometry of the internal representations and the classifier in the final phase of training, after the training error vanishes[38]. Our outcome is different: the predictions for distinct missions become nearly constant after cross-group scaling. Work on predictive uncertainty under shift shows that confidence and calibration can get worse when the deployment data change[39], and two-sample tests can detect a change between reference and incoming feature distributions without target labels[40]. Rubrics for production readiness call for monitors on data, models and serving pipelines rather than a single held-out score[41]. Table 1 sets out how these approaches relate to ours.

Table 1. Comparison with adjacent diagnostics. Label-free inference does not remove the need for development-time threshold calibration and evaluation.

Prior method	Failure detected	Label requirement	Dependence on a reference model	Limitation addressed by our approach
Neural-collapse geometry[38]	Terminal class-feature geometry	Class labels; internal activations	No external model	Targets output constancy from grouped preprocessing, not terminal-phase geometry.
Predictive uncertainty[39]	Uncertainty and calibration under shift	Labels for evaluation	Method-dependent; often ensembles	Directly tests a distinct output degeneracy.
Dataset-shift tests[40]	Source–target distribution change	No target labels for alarm	Reference distribution; sometimes a domain classifier	Tests within-group model outputs, not only input shift.
ML pipeline validation[41]	Broad data, training, serving regressions	Test-dependent	Schema, baseline, or prior version	Supplies a concrete group-level output invariant.
This work	Near-constant within-group predictions	None to compute; labels to calibrate/evaluate	None for spread; optional XGBoost triage	Isolates cross-target pooling and separates collapse from low AUC.

2.4 Research gap

Taken together, the literature offers ways to normalize data well, ways to detect shortcuts and shift, and general advice to monitor pipelines. It does not describe what happens when one scaler is shared across groups of a sequential dataset, why the standard tree baseline cannot see the resulting failure, or how to detect it cheaply. It also does not study learned screens that stop individual Monte Carlo trajectory runs early. This paper addresses those gaps.

3 Methodology

3.1 Overview

Figure 1 summarizes the study. A trajectory simulator produces the missions. A short window of telemetry from the start of each flight is scaled in one of three ways and passed to two models: a Transformer screen and an XGBoost baseline. Three checks are then compared: a pooled AUC over a group of targets, the baseline on the same input, and the spread of the predictions within each target.

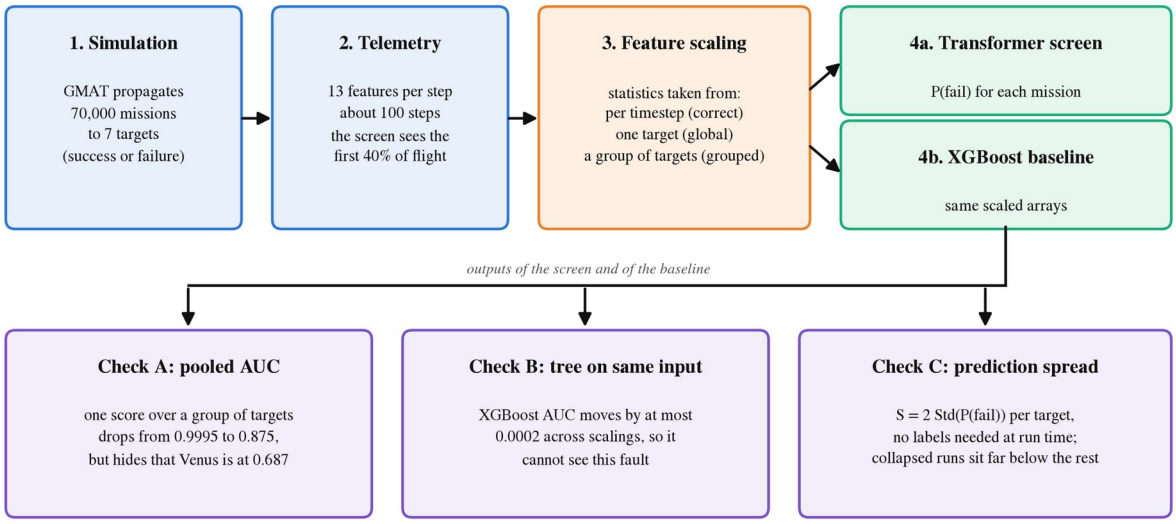


Figure 1. Overview of the study. Missions are simulated, the first 40% of each flight is scaled in one of three ways and passed to a Transformer screen and to an XGBoost baseline on the same scaled values. The three checks at the bottom are compared in Section 4.

3.2 Simulated mission data

The data were generated with NASA's General Mission Analysis Tool (GMAT), an open source program for designing and analyzing space missions. Its propagator, the part that integrates the equations of motion forward in time, has been checked against other established mission tools on a large set of test cases[42], and it is deterministic: the same inputs always give the same trajectory.

We simulated 70,000 missions that leave the Earth, 10,000 to each of seven interplanetary targets: Mercury, Venus, Mars, Jupiter, Saturn, Uranus and Neptune. Each mission starts from a parking orbit around the Earth and then performs a burn that puts the spacecraft on its transfer. Each mission has six injection offsets, which are small random changes to the mission plan. Three are the components of the transfer burn along the velocity, normal and binormal directions (dv_V, dv_N and dv_B). The other three describe the orientation of the parking orbit: the right ascension of the ascending node (RAAN), the argument of periapsis (AOP) and the inclination (INC). Each mission is labeled a success or a failure, and failures are further divided into surface impact, an orbit that is too high, or a missed target. Table 2 gives the size of each target set.

Table 2. The simulated missions of each target. Failure share is the share of missions labeled as failures. Steps is the mean number of telemetry steps per mission after downsampling. Flight time is the mean simulated flight time, which is also the propagation cost that the screen tries to save.

Target	Group	Missions	Failure share	Steps	Flight time (days)
Mercury	inner	10,000	78.5%	102	127
Venus	inner	10,000	66.2%	94	175
Mars	inner	10,000	73.7%	100	311
Jupiter	outer	10,000	65.1%	101	1,197
Saturn	outer	10,000	65.0%	101	2,650
Uranus	outer	10,000	66.1%	101	7,029
Neptune	outer	10,000	65.0%	100	13,419

The transfers are sampled every 54,000 s (15 hours) and then downsampled to about 100 steps per mission, so that sequences from targets whose flight times differ by two orders of magnitude become comparable. Thirteen features are recorded at each step. Ten describe the motion: the position in a rotating (synodic) frame, the specific orbital energy, the flight path angle, the distance to the target divided by the size of its sphere of influence, the radial velocity toward the target, the speed, the distance from the Earth and the eccentricity. Three are constant for a mission and describe the target: the ratio of its gravitational parameter to the Sun’s, the ratio of its sphere of influence to the transfer distance, and the transfer distance in astronomical units. An eighth target, the Moon, was simulated and deliberately left out. It is a six day transfer near the Earth, sampled every 60 s, with neither a cost structure like that of Section 4.7 nor a heliocentric regime.

The missions of each target are split by mission into 70% for training, 15% for validation and 15% for testing. The split is a fixed function of the number of missions and the seed, defined once in the code, so no model is ever scored on data that were used for model selection. This is not a theoretical concern. Two earlier analyses in this project shuffled the same number of

missions with the same seed in separate code, and missions that one model had used for validation ended up in the test set of the other analysis.

3.3 The screening task

The screen observes the first 40% of a mission’s flight and decides whether to stop it. The model outputs P(fail), the probability that the mission fails. A threshold on P(fail), chosen on the validation split, turns this probability into a decision. The value of the screen is the propagation time it saves on failed missions, and its cost is the good missions it stops by mistake.

3.4 Feature scaling

For a feature value x, the scaled value is

$$z = (x - m) / q \tag{1}$$

where m is a center and q is a scale, both estimated on training data only. The global and grouped settings use robust scaling, where m is a median and q is an interquartile range (the distance between the 25th and 75th percentiles). Robust scaling is a common choice because the median and the interquartile range are not moved much by outliers. The per timestep setting uses the mean and the standard deviation instead. The settings differ in which part of the data m and q come from (Fig. 2). Global and grouped use the same kind of statistics, so the difference between those two is the pooling alone.

- Per timestep. For each feature and each timestep index, m and q are the mean and the standard deviation over the training missions of one target at that index. This is the setting used by the working system.
- Global. One median and one interquartile range per feature and target, pooled over all the timesteps of that target.
- Grouped. One median and one interquartile range per feature, pooled over all the timesteps of all the targets in a regime group. The inner group is Mercury, Venus and Mars, and the outer group is Jupiter, Saturn, Uranus and Neptune. This is the setting that failed in our production system.

The grouped setting is not an artificial construction. A single robust scaler fitted to a concatenated training set is what many standard preprocessing tutorials produce. It raises no warning, and the result looks like an ordinary normalized array. Everything else (the

architecture, the seed, the split, the optimizer and the number of epochs) is the same in all three settings.

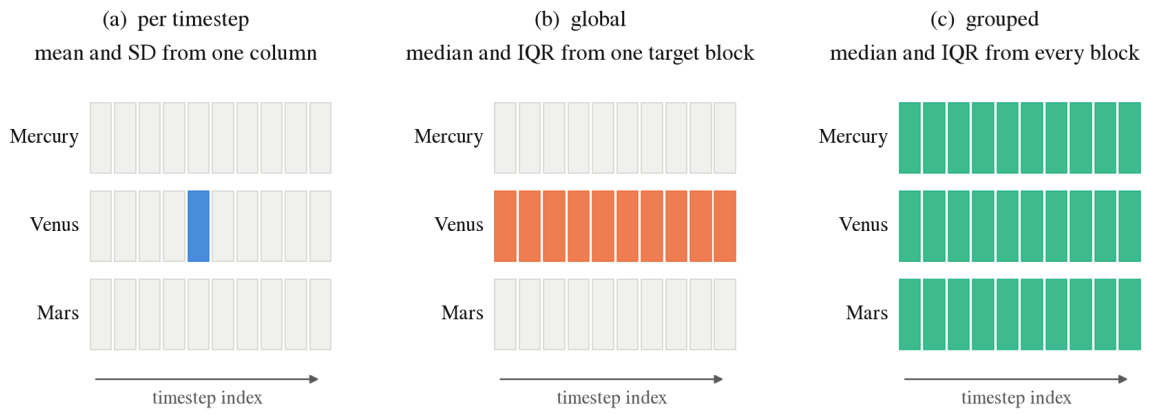


Figure 2. The three scaling settings and the part of the data each one takes its statistics from. Each cell is all the missions of one target at one timestep index, and the shaded cells are the ones the statistics are estimated from. Per timestep scaling uses the mean and the standard deviation, and global and grouped scaling use the median and the interquartile range. Everything else in training is the same.

To measure how much useful variation survives the scaling, we use the signal ratio. It is the average standard deviation of the scaled features across missions, computed at each timestep and averaged over the observed window only. The window matters. Averaged over the entire flight the measure is uninformative: late timesteps have a large spread across missions, because the missions that fail have physically diverged by then, and this spread swamps the early signal, so all settings appear alike. The model commits at 40% of the flight, so the relevant range is the one inside the prefix that the model observes.

3.5 Models

The screen is a Transformer encoder. A Transformer reads the whole sequence at once and uses attention to let every timestep look at every other one, which makes it a strong and widely used model for sequences. Our encoder uses pre-layer normalization (Pre-LN), a model width of 128, 8 attention heads and 4 layers. A learned summary token (CLS) is added to the start of the sequence, and its output is used to make the prediction. Two output heads share this trunk: one predicts whether the mission fails and one predicts the type of failure. The model is trained on prefixes of random length, so that it can give a prediction at any point in the stream and not only at the 40% mark.

The baseline is XGBoost, a gradient boosted ensemble of decision trees, with 300 trees of depth 5. It is fitted to the same scaled values, flattened into one row per mission. Every row

must have the same length, so the tree reads a fixed window: the first 40% of the shortest mission of the target, which is 14 to 40 timesteps, with values clipped to ± 10. It is important that the baseline does not work from a separately designed feature set but from the same scaled values that the network sees. This is what the same input means throughout this paper, and it carries most of the argument. Table 3 lists the training settings.

Table 3. Training settings. The same settings are used for all three scaling modes.

Setting	Value
Encoder	Transformer, pre-layer normalization, CLS token
Width, heads, layers	128, 8, 4 (feed-forward size 512, dropout 0.1)
Output heads	failure probability, and failure type (loss weight 0.5)
Loss	binary cross-entropy weighted for class balance
Optimizer	AdamW, learning rate 0.001, weight decay 0.0001
Schedule	cosine annealing, 60 epochs, batch size 128
Gradient clipping	norm 1.0
Training prefixes	random length, 5% to 40% of each flight
Failure-type sampling	weights proportional to $(1/\text{count})^{0.5}$
Model selection	best validation AUC
Baseline	XGBoost, 300 trees, depth 5, learning rate 0.08
Second dataset	same Transformer, 20 epochs, batch size 256, window and horizon of 30 cycles; XGBoost with its default learning rate

3.6 The prediction spread diagnostic

A collapsed model gives nearly the same P(fail) to every mission, so we look at how much its predictions vary. For a group g with a batch of predictions $\hat{p}_i$, the group spread is the standard deviation of those predictions, $s_g = \text{sd}(\hat{p}_i)$. The score we report is

$$S = 2\, s_g \tag{2}$$

which runs from 0 for a constant output to 1 for the widest spread a probability can have (half of the predictions at 0 and half at 1). It needs no labels at inference time and no second model. Labels are used only during development, to say which runs have collapsed and to set the threshold. A second score divides by the spread of a healthy model trained on the same data:

$$D = s_g / (s_{\text{healthy}} + \varepsilon) \tag{3}$$

It needs a model that was trained the right way, and when something is broken that is exactly what we do not have. We therefore use S as the main score and report D for comparison. We also compare S with three other label-free summaries of the same predictions: the interquartile range, the range, and the mean binary entropy.

To have models in every state, we repeat the ablation on 5 seeds, 4 targets (Venus, Mars, Mercury and Jupiter) and the 3 scaling settings, which gives 60 runs, and keep the predictions on the validation and test missions of each run. The settings give four states. Per timestep runs are healthy, global runs are mildly hurt, grouped runs that keep working are severely hurt, and grouped runs that cannot rank missions are collapsed. A run counts as collapsed if its AUC on the split in question is 0.85 or lower.

The choices in this evaluation were made in the following order, and we state them plainly because the order matters. The group definitions and the three scaling settings come from the ablation. The AUC cutoff of 0.85 was set after the validation and test AUCs of all 60 runs had been seen. It was placed in a gap between the test AUCs (0.82 to 0.895) and between the validation AUCs (0.81 to 0.907), so it was not chosen in advance, and test results influenced it. Given the cutoff, the spread threshold was chosen on validation predictions and validation AUCs only: it is the middle, on a log scale, of the gap between the largest S of a collapsed run and the smallest S of a working run. The test predictions were used afterwards, to count the flagged runs. Because of this order, the test result is a check of the threshold and not an independent test of the cutoff, and Section 4.5 reports how the result changes with the cutoff and when the threshold is set on other seeds.

3.7 Evaluation metrics

The main measure of ranking quality is the area under the receiver operating characteristic curve (AUC). It is the probability that a randomly chosen failed mission receives a higher P(fail) than a randomly chosen successful one, so 0.5 is chance and 1.0 is a perfect ranking. Because AUC depends only on the order of the predictions, it cannot tell a weak model from a constant one on its own, which is why we also report the spread of the predictions. For the screening decision we report the failure recall (the share of failed missions that are stopped), the share of good missions lost (successful missions that are stopped by mistake) and the F1 score of the failure class. The economic measure is the share of propagation time saved, where each mission is charged its propagation time, so outer planet missions weigh much more than inner ones.

3.8 Statistical analysis

Five seeds are few, so we report the individual runs rather than fitted intervals wherever possible. For the comparison of per timestep and grouped scaling on the same test missions we use three tests, computed from the saved predictions without retraining. First, a paired

bootstrap with 2,000 resamples of the test missions gives a 95% interval for each AUC and for their difference. Second, McNemar's exact test on the decisions at P(fail) of 0.5 asks whether the grouped model gets a different set of missions wrong. Third, Welch's t-test compares the test AUCs of the two settings across the five seeds. For the share of seeds on which the collapse occurs we give a 95% Wilson interval.

3.9 Second dataset

To see what happens outside our simulator we used the NASA C-MAPSS turbofan data[43], a standard public set for remaining useful life work[44]. It contains run to failure sensor records for fleets of aircraft engines. Subset FD001 has one operating condition, and FD002 and FD004 have six. The six conditions play the role that the targets play in our data: the sensors sit at very different levels in each one, and the slow drift that shows an engine wearing out is small next to that gap.

The task is to look at the last 30 cycles of an engine and say whether it will fail within the next 30 cycles. The split is by engine, 70/15/15, so no engine is in two sets. We scale the 14 sensors that change over an engine's life in two ways: pooled, with one median and interquartile range for every row, and per group, with one for each operating condition. The model is the Transformer described above with the same optimizer, trained for 20 epochs on 5 seeds, and the baseline is XGBoost on the same scaled window. FD001 has a single condition, so the two scalings are the same there, and it serves as the control.

3.10 Implementation details and use of AI tools

All experiments were run with PyTorch 2.13 and XGBoost 3.4.1 on one NVIDIA GeForce RTX 4060 laptop GPU with 8 GB of memory. One run of the ablation takes between about 40 s and 5 min. Every number in this paper is read from machine-readable result files written by the experiment scripts, and the tables and figures are generated from those files.

During the preparation of this work the authors used Claude (Anthropic) to write and debug the experiment code, to draft the text of all sections, and to check the references and the numbers against the result files. The authors reviewed, edited and verified the content and take full responsibility for it.

4 Experimentation and results

4.1 Normalization ablation

The first result is a controlled decomposition (Table 4). Pooling across time within a single target (the global setting) reduces the signal by up to a factor of 23, and the network still works on all four targets, with a loss of a few thousandths of AUC at most. In contrast, pooling feature statistics across heterogeneous target groups produces substantially more severe degradation, including near-constant predictions for Venus: the output standard deviation is 3.11e-05, which is one value for every mission, and the validation AUC drops from 0.9999 to 0.6037.

Table 4. Normalization ablation. Identical architecture, seed and split; only the pooling differs. 'sig' is the mean within timestep standard deviation of the normalized features in the observed window. 'std' is the standard deviation of P(fail) on unseen missions, the quantity that identifies a collapse, as against a model that is merely inaccurate.

	per-timestep			global			grouped		
Target	**sig**	**AUC**	**std**	**sig**	**AUC**	**std**	**sig**	**AUC**	**std**
Venus	0.7681	0.9999	4.72e-01	0.0518	0.9947	4.50e-01	0.0023	0.6037	3.11e-05
Mars	1.0829	0.9994	4.33e-01	0.0472	0.9972	4.00e-01	0.8005	0.9422	4.06e-01
Mercury	0.7686	0.9994	4.27e-01	0.1321	0.9970	4.23e-01	0.0484	0.9069	4.25e-01
Jupiter	0.7708	1.0000	4.78e-01	0.0763	1.0000	4.78e-01	0.0247	0.9997	4.75e-01

We use the term collapse and not underfitting because of Fig. 3. A low AUC alone is ambiguous, since it is also produced by a model that has learned something poor or wrong. A predicted distribution on unseen missions that is three ten-thousandths wide is not ambiguous: the output no longer depends on the input.

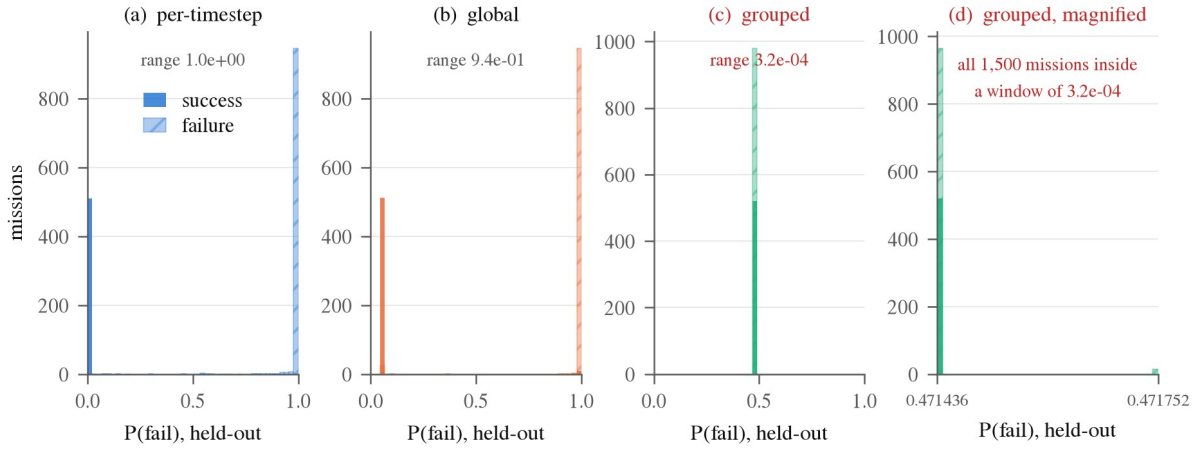


Figure 3. P(fail) on Venus, split out for each setting. Per timestep and global normalization put the two populations at opposite ends of the axis. Under grouped normalization every mission gets essentially the same number. Panel (d) magnifies that spike and shows the full range covers about three ten-thousandths. This is what makes a collapse different from a merely bad model.

The collapse is selective, and we report it together with the cases that do not break. On the remaining three targets the same manipulation leaves the output varying and costs between 0.0003 and 0.0925 AUC. The signal ratio is not a measure of severity either. Jupiter compresses its signal to 0.0247 and Mercury to 0.0484, yet Jupiter loses 0.0003 AUC while Mercury loses 0.0925. One explanation is that the Jupiter task is easily separable, so that even a very weak signal is sufficient. We did not test this directly. Compression is necessary for the collapse but not sufficient, and how hard the underlying task is appears to modulate it. We do not claim that these data support a rule of the form “it collapses when the ratio is below a given value”. The counterexample is visible in Fig. 4.

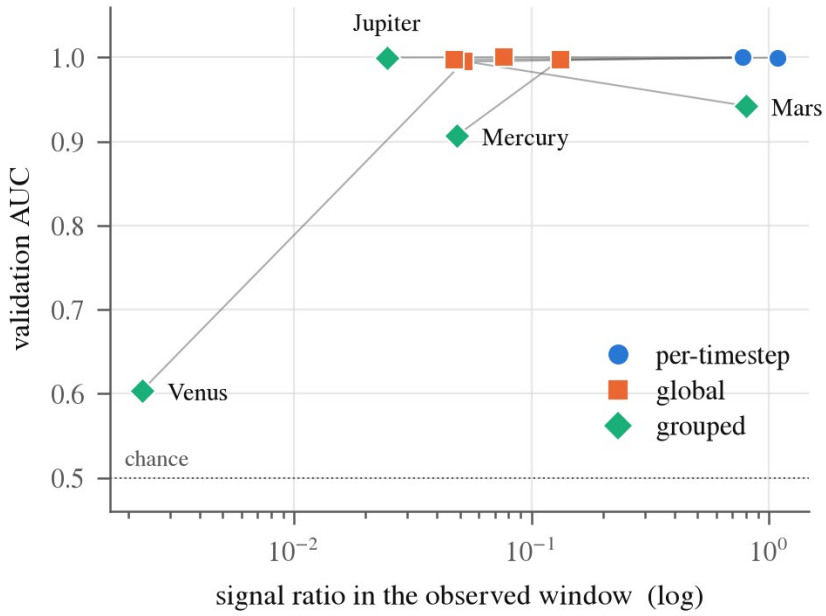


Figure 4. Signal ratio against validation AUC. These points would lie on a curve if the collapse was only due to compression. Jupiter has a lower signal ratio than Mercury and still loses very little.

This ablation isolates normalization and no other part of the model: only the statistics are pooled, and one model is trained for each target. The production failure also shared a single trunk across the group, which makes things worse, since one trunk then has to serve targets whose inputs have all been flattened toward a constant. These numbers are therefore a lower bound on the damage of that configuration. They show that pooled normalization is enough on its own to destroy a target, not that this is a complete reconstruction of the original incident.

4.2 The tree baseline on the same input

Under the grouped setting the XGBoost baseline reports an AUC of 1.0000 on Venus, the target where the network has collapsed to a constant at 0.6037 (Table 5 and Fig. 5). Its AUC

changes by at most 0.0002 across the three settings on all targets, so this scaling choice is not visible to this baseline. The baseline therefore shows that the transformed arrays still contain an ordering that separates the classes. It does not show that the deep model can use the remaining small variation.

Table 5. The tested XGBoost model on the same normalized window for each setting. Its AUC is nearly invariant to the particular rescaling that destroys the network.

Target	per-timestep	global	grouped	spread
Venus	1.0000	1.0000	1.0000	0.0000
Mars	0.9996	0.9998	0.9998	0.0002
Mercury	0.9997	0.9998	0.9998	0.0001
Jupiter	1.0000	1.0000	1.0000	0.0000

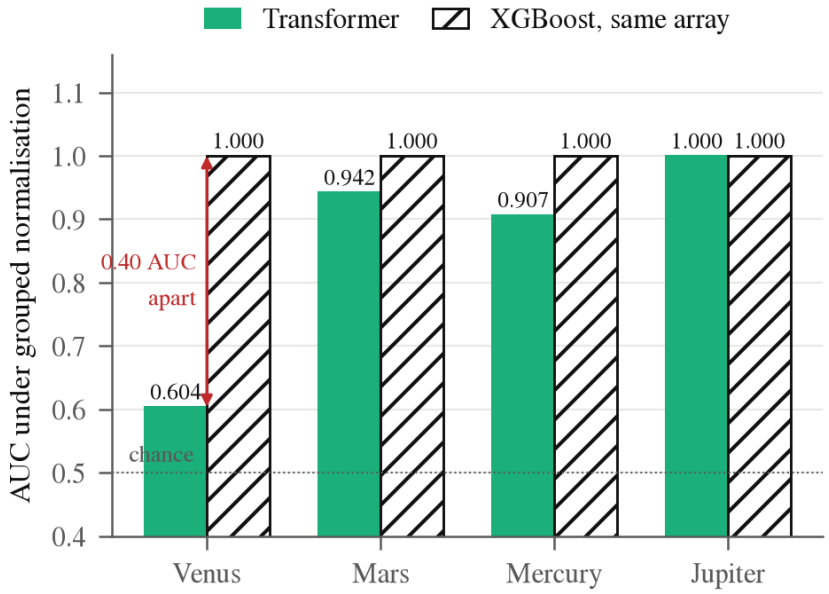


Figure 5. Both models under grouped normalization on the same arrays. The tested XGBoost model is at the ceiling on every target, including the one where the network is just above chance.

The exact claim matters here. The information is present, and this tree can use it. The comparison does not answer the question that has to be answered, which is whether the network can use the compressed signal. Its invariance to this rescaling makes the check blind to this particular defect. That statement is deliberately narrower than a claim that tree baselines cannot find preprocessing errors in general.

4.3 Stability across random seeds

A single run could be unlucky, so we repeated the whole ablation with five seeds. Each seed redraws both the train, validation and test partition and the initialization of the network. The compression does not depend on the seed. The signal ratio for Venus under grouped scaling lies between 0.00222 and 0.00232 across all five runs, a spread of about 4.5% against a

compression of 335 times. The healthy setting is stable as well: the per timestep AUC of two different seeds differs by at most 0.0010, and Mars, Mercury and Jupiter do not collapse on any seed (Table 6 and Fig. 6).

Table 6. Validation AUC across 5 seeds, mean and standard deviation, and the number of seeds on which the grouped run collapsed to a constant output.

Target	per-timestep AUC	grouped AUC	collapsed
Venus	0.9998 ± 0.0001	0.6912 ± 0.1530	4/5
Mars	0.9992 ± 0.0004	0.9457 ± 0.0093	0/5
Mercury	0.9997 ± 0.0002	0.9118 ± 0.0036	0/5
Jupiter	0.9999 ± 0.0002	0.9997 ± 0.0003	0/5

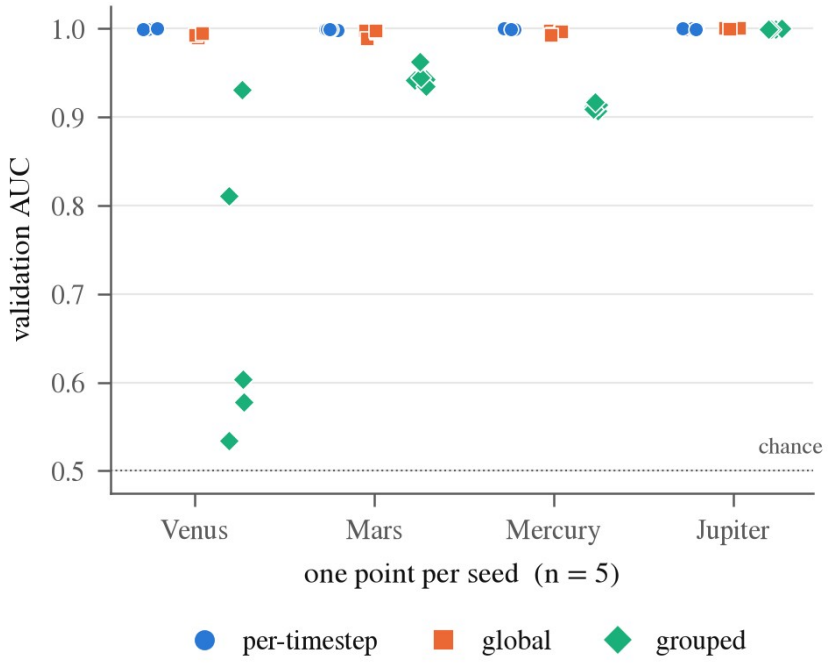


Figure 6. Every seed plotted on its own rather than summarized as an interval. With n = 5 the spread of the raw points is the honest summary.

Whether the network escapes does depend on the seed. Venus falls below a spread of 10^{-3} on 4 of 5 seeds, with a prediction spread on unseen missions between 1.57e-05 and 4.84e-05, four orders of magnitude below the 4.66e-01 of correct normalization. On the remaining seed it does not collapse, and it does not recover either: its prediction spread of 3.64e-02 is still an order of magnitude short, at an AUC of 0.9305 against a healthy 0.9999. The observed recurrence of 4 out of 5 has a 95% Wilson interval of 0.376 to 0.964. That wide interval describes how often the collapse recurs under these seeds. It is not an interval for the sensitivity of a detector, and it does not support universal reliability.

This seed-to-seed variation is the main reason for the spread diagnostic. The grouped AUC of Venus ranges from 0.5338 to 0.9305 across these runs, which overlaps the AUC of a merely

weakened target: Mercury lies inside that range on every seed while still telling missions apart. AUC alone therefore cannot separate the two states, while the prediction spread can. The invariance of the baseline is also repeated. Across 5 seeds, 4 targets and 3 settings, which is 60 fitted XGBoost models, the AUC stays between 0.9989 and 1.0000, and within each seed and target it changes by at most 0.0003 between settings.

4.4 Statistical significance

Table 7 compares per timestep and grouped scaling on the same test missions. On seed 42 the bootstrap interval of the AUC difference excludes zero for Venus, Mars and Mercury, and McNemar's test shows that the grouped model gets a different set of missions wrong on those three targets. For Venus, the grouped model is wrong on 974 test missions that the per timestep model gets right, against 9 the other way. For Jupiter the difference is small, and its interval touches zero. Over all 20 pairs of target and seed, the bootstrap interval of the AUC difference excludes zero in 16 and McNemar's test is significant in 16. The two tests agree on 18 pairs (both significant on 15, and neither on 3 Jupiter pairs), and the two pairs where they disagree are also Jupiter runs. Across the five seeds, Welch's t-test separates the two settings for Mars, Mercury and Venus, and not for Jupiter. These tests confirm that the damage is real and not a chance result of one split. With five runs per setting, however, they say nothing about how often such a collapse would occur in other pipelines.

Table 7. Per timestep against grouped scaling on the same test missions. AUC values and 95% bootstrap intervals (2,000 resamples) are for seed 42. McNemar gives the number of missions that only the per timestep model gets right and the number that only the grouped model gets right, with the exact p value. Welch gives t and p for the test AUC across the five seeds.

Target	Per timestep AUC	Grouped AUC	Difference	McNemar	Welch
Venus	0.9998 (0.9996 to 0.9999)	0.5988 (0.5685 to 0.6280)	0.4010 (0.3717 to 0.4313)	974/9, <0.001	4.05, 0.015
Mars	0.9997 (0.9992 to 0.9999)	0.9418 (0.9306 to 0.9528)	0.0578 (0.0468 to 0.0691)	157/9, <0.001	10.74, <0.001
Mercury	0.9993 (0.9987 to 0.9998)	0.9130 (0.8989 to 0.9264)	0.0863 (0.0730 to 0.1002)	191/1, <0.001	21.97, <0.001
Jupiter	1.0000 (1.0000 to 1.0000)	0.9990 (0.9977 to 1.0000)	0.0010 (0.0000 to 0.0023)	5/0, 0.062	0.04, 0.971

4.5 Evaluating the prediction spread check

The four collapsed runs are exactly the four that fall below the earlier, stricter rule of a group spread under 10^{-3}. At the AUC cutoff of 0.85, the spread threshold chosen on validation is 2.7e-03, in the middle of the gap between the largest S of a collapsed run (1.0e-04) and the smallest S of a working run (0.071). Applied unchanged to the test predictions, it flags 4 runs

out of 60, which are the 4 with a test AUC of 0.85 or lower, and no working run (Table 8 and Fig. 7). The score *D*, with its threshold chosen the same way, flags the same 4, so the reference model did not add anything here.

Table 8. The 60 runs by state. *S* is the spread score on the test missions, and AUC is the median on the test missions. The spread of the collapsed runs is at least 750 times smaller than that of any other run.

State	Runs	*S* (smallest to largest)	Median AUC
Healthy	20	0.807 to 0.963	0.9996
Mild	20	0.748 to 0.962	0.9955
Severe	16	0.073 to 0.961	0.9430
Collapsed	4	3.15e-05 to 9.69e-05	0.5912

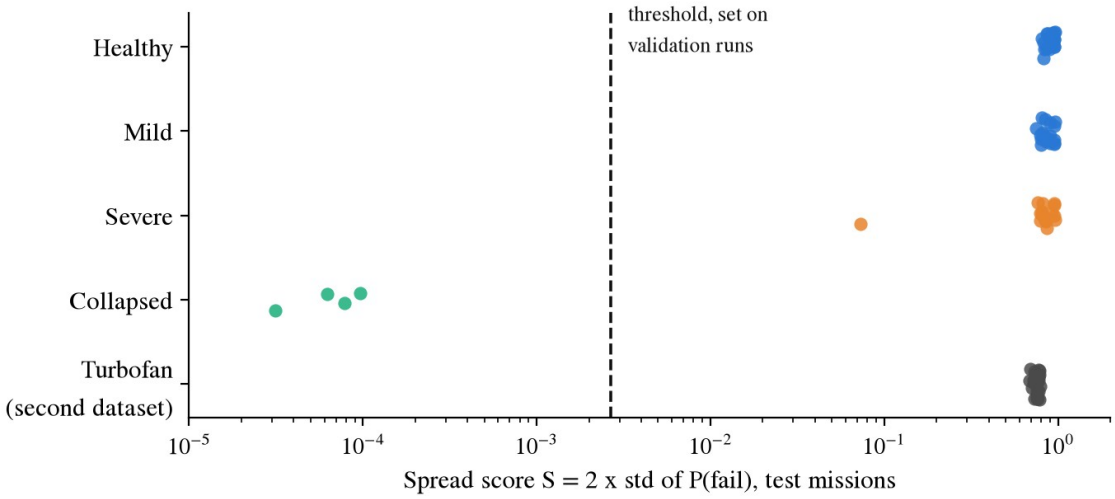


Figure 7. The spread score of every run on the test missions. The dashed line is the threshold chosen on the validation runs. The turbofan runs of Section 4.6 are shown for comparison, and none is flagged.

The score is meant for collapse, not for every loss of accuracy. The Mercury runs under grouped scaling lose about 0.09 AUC while their spread stays above 0.75, so the check does not flag them. Venus on seed 3 is the hard case. Its *S* is 0.073, about ten times below a healthy run, and it still ranks missions at an AUC of 0.92. It is above the threshold and is not flagged.

Table 9 shows how the result depends on the AUC cutoff. A single threshold can be set on the validation runs only at cutoffs of 0.85 and 0.90. Below 0.85 the Venus run on seed 2, with a validation AUC of 0.81, is labeled as working, but its spread (7.9e-05) lies inside the range of the collapsed runs, so no threshold separates the two labeled groups. At 0.95 fourteen runs count as collapsed, including Mercury runs that keep their spread, and the groups overlap again. The ranking is more stable than the threshold: the ROC AUC of *S* on the test predictions is 1.000 for every cutoff from 0.60 to 0.85 and 0.993 at 0.90. The score separates

runs with a near-constant output from the others. Whether a run with a constant output but a usable ranking, such as Venus on seed 2, is called collapsed depends on the cutoff.

Table 9. Dependence on the AUC cutoff that defines a collapsed run. Collapsed runs are counted on validation and on test. The threshold is set on validation only, and 'none' means that the labeled groups overlap on validation. Missed runs and false alarms are counted on test. The last four columns give the ROC AUC, on test, of four label-free summaries of the predictions.

Cutoff	Collapsed (val/test)	Threshold	Missed	False alarms	*S*	IQR	Range	Entropy
0.60	2/3	none	n/a	n/a	1.000	0.988	0.994	0.982
0.70	3/3	none	n/a	n/a	1.000	0.988	0.994	0.982
0.80	3/3	none	n/a	n/a	1.000	0.988	0.994	0.982
0.85	4/4	2.7e-03	0	0	1.000	1.000	1.000	1.000
0.90	4/5	2.7e-03	1	0	0.993	0.945	0.978	0.989
0.95	14/14	none	n/a	n/a	0.879	0.787	0.984	0.972

To test the threshold on seeds it was not chosen on, we set it on the validation predictions of four seeds and applied it to the test predictions of the fifth, for each seed in turn. At a cutoff of 0.85 this gives 4 collapsed runs, 0 missed and 0 false alarms over the 60 runs. At 0.90 there are 5 collapsed runs, 1 missed (a Mercury run that keeps its spread) and 0 false alarms. At the cutoffs from 0.65 to 0.80, no threshold could be set on three of the seeds, for the reason given above, and on the other two there was 1 missed run and 1 false alarm. These seeds redraw the split and the initialization, but they use the same four targets, so this is not a test on new targets.

The statistic is an ordinary output variance, and we do not claim that it is the best summary. Table 9 also gives the ROC AUC of three other label-free summaries of the same predictions. Up to a cutoff of 0.85 all of them separate the collapsed runs well (0.98 to 1.00). At 0.90 the spread score is the highest (0.993, against 0.989 for the entropy, 0.978 for the range and 0.945 for the interquartile range), and at 0.95 the range and the entropy are higher than it. The differences are small and rest on four collapsed runs from one target. What these results support is that a summary of the predictions computed per group detects a failure that the tested baseline does not detect and that a pooled score shows only in part. They do not support a claim that this score is better than other output summaries. We also tried a fourth summary, the share of predictions close to the median. It did as well as the others up to 0.85, but fell to 0.818 at 0.90 and 0.293 at 0.95, so we do not use it.

A pooled score shows the collapse only in part. For the inner group (Mercury, Venus and Mars) the AUC over the test missions of all three targets is 0.9995 with per timestep scaling and 0.875 with grouped scaling, averaged over the five seeds. That is a visible drop. It is, however, 0.19 above the AUC of Venus alone (0.687) and above the mean of the three per

target AUCs (0.845), because a model that is constant within a group still ranks the groups against each other. The pooled value also does not say which target failed. The spread is computed per target, and it is 3.1e-05 to 9.7e-05 for the four collapsed Venus runs, against 0.7 or more for Mercury and Mars under grouped scaling.

The spread is computed over a batch of predictions, so it depends on the batch size and on how the probabilities are calibrated. We resampled the saved test predictions at random. Batches of 30 or more missions flagged every collapsed run and no working run. Batches of 10 flagged 1.1% of the working batches, and batches of 5 flagged 7.4%. These batches contain both outcomes, because they were drawn at random from the test missions. A batch that holds missions of one outcome only, or from one narrow region of the inputs, was not tested, and a small spread would be expected there. We also rescaled the predicted log-odds by a temperature between 0.25 and 4. The smallest spread of a working run then moved between 0.28 and 0.018, and the threshold of 2.7e-03 still separated the 4 collapsed runs from the 56 working runs. This is a monotone change of the outputs of one model family. A model with different calibration, architecture, label prevalence or batch size needs its own threshold.

The signal ratio can be measured before any training, so it can serve as an early warning. Every collapsed run has a signal ratio of 0.0022 or 0.0023. Five runs are below 0.01, namely the 4 collapsed runs and Venus on seed 3, and no run at 0.0243 or above collapsed. The ratio can therefore warn that a configuration is at risk, but it cannot say which of the runs will collapse. For that, the spread after training is needed.

4.6 Second dataset

On the turbofan data, pooling shrinks the signal about nine times, from 0.802 to 0.083 on FD002 and from 0.859 to 0.097 on FD004. The model does not break (Table 10). On both subsets the pooled Transformer is slightly ahead of the per group one, the spread of its predictions is the same, and the tree performs well either way. The spread check, with the threshold it already had, flags none of the 30 runs.

Table 10. Turbofan data, 5 seeds. ‘signal’ is the mean within group standard deviation of the scaled features. AUC is on test engines, mean and standard deviation. *S* is the spread score of the Transformer.

Subset	Scaling	signal	Transformer AUC	Tree AUC	*S*
FD001	pooled	0.784	0.9916 ± 0.0017	0.9953	0.760
FD001	per-group	0.784	0.9916 ± 0.0017	0.9953	0.760
FD002	pooled	0.083	0.9898 ± 0.0014	0.9862	0.767
FD002	per-group	0.802	0.9855 ± 0.0025	0.9925	0.755
FD004	pooled	0.097	0.9910 ± 0.0010	0.9890	0.734
FD004	per-group	0.859	0.9881 ± 0.0044	0.9933	0.731

This agrees with the ablation. Shrinking the signal is not enough to break a model: in our data Mercury and Jupiter lose most of their signal under grouped scaling and keep working, and here the drift is still present after pooling and the model finds it. It also shows that the check does not raise an alarm on a second kind of data. It does not show that the threshold carries over to other domains.

4.7 Where the screen belongs

The final question is whether a sequence model is the right tool for this screen at all. A classifier that uses only the six injection offsets, and is applied before any propagation, saves 64.5% of the compute, against 38.3% for the telemetry Transformer at 40% of the flight, although it loses more good missions, 0.76% against 0.21% (Table 11). Accuracy also levels off between 10% and 40% of the flight. This is not a modeling fault. The simulator is deterministic, so the outcome is a fixed function of the injection offsets, and the trajectory is an integral of those offsets. It cannot carry information that the offsets do not have.

Table 11. Screening economics. Compute is charged in propagation days, which span about a 100x range across the seven targets. Thresholds are fitted on validation and reported on the untouched test split, so the recall column is measured out of sample rather than achieved by construction.

Screen	Compute saved	Good missions lost	Failure recall
T0: six launch parameters, before propagating	64.5%	0.76%	99.06%
T40: telemetry Transformer at 40%	38.3%	0.21%	98.87%
Cascade: T0 where confident, else T40	65.0%	1.62%	99.94%

This does not make the task trivial. Logistic regression on the same six offsets reaches only 0.4824 to 0.5410 AUC on the seven targets, which is chance, so the mapping from the offsets to the outcome is highly nonlinear and needs a learned model. It does not need a temporal one. A sequential screen is worth using only when the outcome is not fixed at injection: with stochastic effects during flight, unmodeled dynamics, sensor noise, or campaigns where the launch parameters are unknown. That is the setting that reinforcement learning trajectory work faces when it certifies policies through dispersed Monte Carlo campaigns[12]. For the T0 screen, a five-split analysis (seeds 0 to 4) gives compute saved of 64.50% ± 0.61 percentage points, good missions lost of 1.24% ± 0.24, and failure recall of 98.99% ± 0.18 (mean and standard deviation across five splits). The T40 and cascade rows remain single seed

estimates. Five correlated resplits of one synthetic corpus do not measure uncertainty across campaigns or domains.

5 Discussion

5.1 What the results mean for mission assurance

The first paragraph below states what the experiments in this paper show. The second and third describe consequences and a workflow that have not been tested.

Demonstrated result. Within the GMAT experiment, the per target spread of the predictions exposes four near-constant Venus runs that the tested XGBoost comparison does not identify and that a pooled AUC shows only in part. Because it checks the behavior at the interface between preprocessing and model, the same kind of check can support verification and validation in other simulation screening pipelines: it tests whether changing the candidate inputs still changes the output of the learned screen. It does not validate the trajectory propagator, prove physical correctness, certify flight software, or establish a threshold outside the evaluated configuration.

Operational implication. If a similar collapse went unnoticed in a mission design campaign, the screen could assign nearly the same decision to very different candidates. Depending on that constant and on the decision threshold, it could discard viable trajectories, keep expensive failures, distort estimates of the size of the feasible region, and bias trade studies or robustness margins. A healthy score from other target groups could hide the affected group and create false confidence in the campaign. These consequences are analyses of possible failures, not outcomes measured in a flight campaign.

Proposed pipeline. During development, a team would generate labeled working, degraded and deliberately broken validation states for each combination of dataset, architecture, target group and preprocessing version. It would set a group-specific spread threshold and a minimum batch size on those states, freeze both before the untouched test evaluation, and record false alarm and miss rates with their uncertainty. At run time, each screening batch would log the versions of the model, scaler, data and groups, compute the group spread alongside input-signal and feature-shift checks, and compare only with the matching frozen limits. An alarm would set the learned screen aside, keep all candidates, and fall back to full propagation rather than stopping trajectories. A tree comparison can help offline diagnosis by asking whether information remains in the same arrays, but it is not needed to compute the run-time statistic.

5.2 Limitations

- Seeds. Replication is limited to 5 seeds. The observed recurrence of 4 out of 5 has a wide 95% Wilson interval (0.376 to 0.964), and five seeds rule out neither an unknown controlling factor nor substantial uncertainty.
- Threshold. The AUC cutoff that defines a collapsed run (0.85) was set after the validation and test AUCs had been seen. The spread threshold (2.7e-03) was chosen on validation predictions only, so the test result is a check of that threshold and not an independent test of the cutoff. A single threshold can be set only for cutoffs of 0.85 and 0.90. All 4 collapsed runs are Venus, so one kind of collapse has been tested. The threshold is not expected to transfer unchanged to another dataset, architecture, calibration, target group, prevalence or batch size, and batches with one outcome class were not tested.
- Datasets. The collapse is shown on the GMAT corpus. The turbofan data show the same loss of signal without a collapse and without an alarm. That supports the absence of false alarms there, but it does not show that the threshold carries over to other domains.
- Targets. One of four targets collapses, and all collapsed runs are Venus. Other target combinations and group definitions are needed to estimate how much the composition of a group matters.
- Architecture. An LSTM control on the ordinary, correctly preprocessed screening task reaches a test AUC of 0.9928 against 0.9958 for the Transformer, so the base task is not unique to one architecture. The LSTM was not run under grouped scaling, so whether the collapse occurs beyond the Transformer remains open.
- Severity. The check flags near-constant output. It does not flag a model that loses accuracy but keeps its spread, such as Mercury, and graded compression sweeps were not built.
- Synthetic and deterministic data. There are no execution errors, no unmodeled accelerations and no sensor noise, and all missions of a target share a common time base. That common time base is what makes per timestep scaling possible, and it is a property of the generator rather than of a real campaign. Public satellite telemetry benchmarks[17,19] are a natural next place to test the check.

6 Conclusion and future work

When one scaler is pooled across groups with different scales, the tested Transformer can turn into a constant for one group, while the tested scale-invariant XGBoost baseline still finds the same arrays separable and a pooled score shows the damage only in part. None of

the pieces here are unusual. The defect enters through a routine preprocessing step, the model is a plain Transformer, and the baseline is the one most practitioners would choose. The conclusion is not that trees cannot diagnose preprocessing errors, but that a check which is invariant to this transformation cannot answer whether a scale-sensitive network can use the compressed signal.

We do not claim that the fix, normalizing each group separately, is new. The lesson is about detection: monitor the behavior of the outputs within each group, in addition to pooled scores and baselines. Computing the prediction spread needs no labels at run time, but turning it into an alarm needs a threshold set for each model version and an independent labeled evaluation during development. On a second dataset, where pooling shrank the signal and the model survived, the same check raised no alarm.

Future work should test the check on real telemetry, on other architectures such as LSTMs trained under grouped scaling, on more target groups and group definitions, and on graded levels of signal compression, with the threshold fixed before any test data are seen. For trajectory screening in particular, a screen based on the injection parameters should be preferred whenever the simulator is deterministic, and a sequential screen should be reserved for campaigns in which the outcome is not fixed at injection.

Data availability

The simulated mission data were generated with NASA’s General Mission Analysis Tool. The per target extracts used in the experiments can be regenerated with the scripts in the code repository and are available from the corresponding author on reasonable request. The full mission table (about 71 GB) is too large to share in a public repository. The NASA C-MAPSS turbofan engine degradation data are publicly available from the NASA Prognostics Center of Excellence data set repository.

Code availability

The code for all experiments, the result files from which every number is read, and the code that produces the tables and figures are available at https://github.com/Haise-727/gmat-pred.

References

1. Nickel, C., Dichmann, D. J., Lebois, R., Lutz, S. & Parker, J. J. K. Monte Carlo Analysis as a Trajectory Design Driver for the TESS Mission. In *AIAA/AAS Astrodynamics Specialist Conference* (2016). https://doi.org/10.2514/6.2016-5665

2. Massari, M., Di Lizia, P. & Rasotto, M. Nonlinear Uncertainty Propagation in Astrodynamics Using Differential Algebra and Graphics Processing Units. *Journal of Aerospace Information Systems* **14**, 493–503 (2017). https://doi.org/10.2514/1.I010535
3. Jia, B. & Xin, M. Data-Driven Surrogate Model and Admissible-Region-Based Orbital Uncertainty Propagation. *Journal of Aerospace Information Systems* **19**, 753–770 (2022). https://doi.org/10.2514/1.I011107
4. Yin, J., Wang, F. & Shankar, M. Strategies for Integrating Deep Learning Surrogate Models with HPC Simulation Applications. In *IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW)* (2022). https://doi.org/10.1109/IPDPSW55747.2022.00222
5. Izzo, D., Märtens, M. & Pan, B. A survey on artificial intelligence trends in spacecraft guidance dynamics and control. *Astrodynamics* **3**, 287–299 (2019). https://doi.org/10.1007/s42064-018-0053-6
6. Izzo, D. et al. Optimality principles in spacecraft neural guidance and control. *Science Robotics* **9**, eadi6421 (2024). https://doi.org/10.1126/scirobotics.adi6421
7. Sola, J. & Sevilla, J. Importance of input data normalization for the application of neural networks to complex industrial problems. *IEEE Transactions on Nuclear Science* **44**, 1464–1468 (1997). https://doi.org/10.1109/23.589532
8. Huang, L. et al. Normalization Techniques in Training DNNs: Methodology, Analysis and Application. *IEEE Transactions on Pattern Analysis and Machine Intelligence* **45**, 10173–10196 (2023). https://doi.org/10.1109/TPAMI.2023.3250241
9. Sánchez-Sánchez, C. & Izzo, D. Real-Time Optimal Control via Deep Neural Networks: Study on Landing Problems. *Journal of Guidance, Control, and Dynamics* **41**, 1122–1135 (2018). https://doi.org/10.2514/1.G002357
10. Li, H., Chen, S., Izzo, D. & Baoyin, H. Deep networks as approximators of optimal low-thrust and multi-impulse cost in multitarget missions. *Acta Astronautica* **166**, 469–481 (2020). https://doi.org/10.1016/j.actaastro.2019.09.023
11. Izzo, D., Sprague, C. & Tailor, D. Machine learning and evolutionary techniques in interplanetary trajectory design. *Modeling and Optimization in Space Engineering*, 191–210 (2019). https://doi.org/10.1007/978-3-030-10501-3_8
12. Zavoli, A. & Federici, L. Reinforcement Learning for Robust Trajectory Design of Interplanetary Missions. *Journal of Guidance, Control, and Dynamics* **44**, 1440–1453 (2021). https://doi.org/10.2514/1.G005794
13. Peng, H. & Bai, X. Machine Learning Approach to Improve Satellite Orbit Prediction Accuracy Using Publicly Available Data. *The Journal of the Astronautical Sciences* **67**, 762–793 (2020). https://doi.org/10.1007/s40295-019-00158-3
14. Balducci, M., Jones, B. & Doostan, A. Orbit uncertainty propagation and sensitivity analysis with separated representations. *Celestial Mechanics and Dynamical Astronomy* **129**, 105–136 (2017). https://doi.org/10.1007/s10569-017-9767-7
15. Naik, K. R., Holmgren, A. & Kenworthy, J. Using Machine Learning to Automatically Detect Anomalous Spacecraft Behavior from Telemetry Data. In *IEEE Aerospace Conference* (2020). https://doi.org/10.1109/AERO47225.2020.9172726
16. Jiang, S., Jiang, Y., Wang, Y., Zhang, X. & Zhang, Z. Anomaly Detection in Spacecraft Telemetry Data Based on Transformer-LSTM. In *International Conference on Intelligent Communication and Networking (ICN)* (2023). https://doi.org/10.1109/ICN60549.2023.10426290
17. Hundman, K., Constantinou, V., Laporte, C., Colwell, I. & Söderström, T. Detecting Spacecraft Anomalies Using LSTMs and Nonparametric Dynamic Thresholding. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 387–395 (2018). https://doi.org/10.1145/3219819.3219845

18. Herrmann, L., Bieber, M., Verhagen, W. J. C., Cosson, F. & Santos, B. F. Unmasking overestimation: a re-evaluation of deep anomaly detection in spacecraft telemetry. *CEAS Space Journal* **16**, 225–237 (2024). https://doi.org/10.1007/s12567-023-00529-5

19. Kotowski, K. et al. European Space Agency Benchmark for Anomaly Detection in Satellite Telemetry. *arXiv preprint arXiv:2406.17826* (2024). https://doi.org/10.48550/arXiv.2406.17826

20. Uriot, T. et al. Spacecraft Collision Avoidance Challenge: design and results of a machine learning competition. *Astrodynamics* **6**, 121–140 (2021). https://doi.org/10.1007/s42064-021-0101-5

21. Acciarini, G. et al. Kessler: A Machine Learning Library for Spacecraft Collision Avoidance. In *8th European Conference on Space Debris* (2021).

22. Stevenson, E., Rodríguez-Fernández, V., Urrutxua, H. & Camacho, D. Benchmarking deep learning approaches for all-vs-all conjunction screening. *Advances in Space Research* **72**, 2660–2675 (2023). https://doi.org/10.1016/j.asr.2023.01.036

23. Ioffe, S. & Szegedy, C. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. In *International Conference on Machine Learning (ICML)*, 448–456 (2015). https://doi.org/10.48550/arXiv.1502.03167

24. Ba, J. L., Kiros, J. R. & Hinton, G. E. Layer Normalization. *arXiv preprint arXiv:1607.06450* (2016). https://doi.org/10.48550/arXiv.1607.06450

25. Wu, Y. & He, K. Group Normalization. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 3–19 (2018). https://doi.org/10.1007/978-3-030-01261-8_1

26. Ulyanov, D., Vedaldi, A. & Lempitsky, V. Instance Normalization: The Missing Ingredient for Fast Stylization. *arXiv preprint arXiv:1607.08022* (2016). https://doi.org/10.48550/arXiv.1607.08022

27. Deng, J., Chen, X., Jiang, R., Song, X. & Tsang, I. W. ST-Norm: Spatial and Temporal Normalization for Multi-variate Time Series Forecasting. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining*, 269–278 (2021). https://doi.org/10.1145/3447548.3467330

28. Passalis, N., Tefas, A., Kanniainen, J., Gabbouj, M. & Iosifidis, A. Deep Adaptive Input Normalization for Time Series Forecasting. *IEEE Transactions on Neural Networks and Learning Systems* **31**, 3760–3765 (2020). https://doi.org/10.1109/TNNLS.2019.2944933

29. Geirhos, R. et al. Shortcut Learning in Deep Neural Networks. *Nature Machine Intelligence* **2**, 665–673 (2020). https://doi.org/10.1038/s42256-020-00257-z

30. Shah, H., Tamuly, K., Raghunathan, A., Jain, P. & Netrapalli, P. The Pitfalls of Simplicity Bias in Neural Networks. In *Advances in Neural Information Processing Systems (NeurIPS)* **33**, 9573–9585 (2020).

31. Lapuschkin, S. et al. Unmasking Clever Hans predictors and assessing what machines really learn. *Nature Communications* **10**, 1096 (2019). https://doi.org/10.1038/s41467-019-08987-4

32. Chaudhary, D. K., Srivastava, S. & Kumar, V. A Review on Hidden Debts in Machine Learning Systems. In *Second International Conference on Green Computing and Internet of Things (ICGCIoT)*, 619–624 (2018). https://doi.org/10.1109/ICGCIOT.2018.8753081

33. Chen, T. & Guestrin, C. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794 (2016). https://doi.org/10.1145/2939672.2939785

34. Grinsztajn, L., Oyallon, E. & Varoquaux, G. Why do tree-based models still outperform deep learning on tabular data?. In *Advances in Neural Information Processing Systems (NeurIPS)* **35**, 507–520 (2022). https://doi.org/10.48550/arXiv.2207.08815

35. Shwartz-Ziv, R. & Armon, A. Tabular Data: Deep Learning is Not All You Need. *Information Fusion* **81**, 84–90 (2022). https://doi.org/10.1016/j.inffus.2021.11.011

36. Ke, G. et al. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems (NeurIPS)* **30**, 3146–3154 (2017).

37. Pasaribu, J., Yudistira, N. & Mahmudy, W. F. Tabular Data Classification and Regression: XGBoost or Deep Learning With Retrieval-Augmented Generation. *IEEE Access* **12**, 191719–191732 (2024). https://doi.org/10.1109/ACCESS.2024.3518205

38. Papyan, V., Han, X. Y. & Donoho, D. L. Prevalence of Neural Collapse during the Terminal Phase of Deep Learning Training. *Proceedings of the National Academy of Sciences* **117**, 24652–24663 (2020). https://doi.org/10.1073/pnas.2015509117

39. Ovadia, Y. et al. Can You Trust Your Model's Uncertainty? Evaluating Predictive Uncertainty under Dataset Shift. In *Advances in Neural Information Processing Systems (NeurIPS)* **32** (2019).

40. Rabanser, S., Günnemann, S. & Lipton, Z. C. Failing Loudly: An Empirical Study of Methods for Detecting Dataset Shift. In *Advances in Neural Information Processing Systems (NeurIPS)* **32** (2019).

41. Breck, E., Cai, S., Nielsen, E., Salib, M. & Sculley, D. The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction. In *2017 IEEE International Conference on Big Data*, 1123–1132 (2017). https://doi.org/10.1109/BigData.2017.8258038

42. Hughes, S. P., Qureshi, R. H., Cooley, D. S. & Parker, J. J. K. Verification and Validation of the General Mission Analysis Tool (GMAT). In *AIAA/AAS Astrodynamics Specialist Conference* (2014). https://doi.org/10.2514/6.2014-4151

43. Saxena, A., Goebel, K., Simon, D. & Eklund, N. Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation. In *International Conference on Prognostics and Health Management*, 1–9 (2008). https://doi.org/10.1109/PHM.2008.4711414

44. Li, X., Ding, Q. & Sun, J.-Q. Remaining Useful Life Estimation in Prognostics Using Deep Convolution Neural Networks. *Reliability Engineering & System Safety* **172**, 1–11 (2018). https://doi.org/10.1016/j.ress.2017.11.021

Funding

The authors received no funding for this work.

Author contributions

Harsha Vardan M Sakamuri: conceptualization, methodology, software, validation, formal analysis, investigation, data curation, writing of the original draft, review and editing, visualization. Rohit Michael Raj: conceptualization, methodology, software, validation, formal analysis, investigation, data curation, writing of the original draft, review and editing, visualization. Valarmathi P.: supervision.

Competing interests

The authors declare no competing interests.